
Informe de Trabajo Práctico 1

Estrategias - Mochila

Materia: Teoría de Algoritmos

Cátedra: Echevarría

Cuatrimestre: Segundo Cuatrimestre

Integrantes

Alumno, Padrón: Francisco Oscar Ballerio, 114777

Alumno, Padrón: Giuliana Sol Tarquiola Coaquira, 110858

Alumno, Padrón: Federico Fredotovich, 112537

Alumno, Padrón: Rafael Monges Lucas, 114754

Fecha de Entrega: 4 de octubre de 2026

Índice

1. Definición Formal del Problema	3
1.1. Supuestos, Limitaciones y Condiciones Generales	3
2. Fuerza Bruta vs. Backtracking	3
2.1. Diseño de los Algoritmos y Estructuras de Datos	3
2.1.1. Fuerza Bruta	3
2.1.2. Backtracking	4
2.1.3. Estructuras de Datos Utilizadas	5
2.2. Seguimiento con Set de Datos Reducido	5
2.2.1. Seguimiento 1: Fuerza Bruta	5
2.2.2. Seguimiento 2: Backtracking	6
2.3. Análisis de Complejidad Temporal	7
2.4. Sets de Datos Utilizados y Criterios	7
2.5. Resultados Obtenidos y Tiempos de Ejecución	8
2.6. Informe de Resultados y Conclusiones	8
3. Greedy	8
3.1. Supuestos, Limitaciones y Condiciones Generales	8
3.2. Diseño del Algoritmo y Estructuras de Datos	9
3.2.1. Garantía de Calidad y Elemento Crítico	9
3.2.2. Pseudocódigo	9
3.2.3. Estructuras de Datos Utilizadas	10
3.3. Seguimiento con Set de Datos Reducido	10
3.4. Análisis de Complejidad Temporal	11
3.5. Sets de Datos Utilizados y Criterios	11
3.6. Resultados Obtenidos y Tiempos de Ejecución	11
3.7. Informe de Resultados y Conclusiones	12
4. Programación Dinámica	13
4.1. Diseño del Algoritmo	13
4.1.1. Variante Alternativa del Problema	13
4.1.2. Principio de Optimalidad y Subestructura Óptima	13
4.1.3. Ecuación de Recurrencia	13
4.1.4. Ecuación de Recurrencia Alternativa	14
4.2. Pseudocódigo y Estructuras de Datos	14
4.3. Complejidad	15
4.3.1. Complejidad Temporal	15
4.3.2. Complejidad Espacial	15
4.4. Seguimiento del Problema Original	15
4.5. Comparación de Tiempos de Ejecución	16
4.5.1. Planteo Tradicional	16
4.5.2. Planteo Alternativo	17
4.5.3. Comparación y Conclusión	18

5. Programación Lineal	18
5.1. Diseño del Algoritmo y Estructuras de Datos	18
5.1.1. Modelo de Programación Lineal Entera	18
5.1.2. Pseudocódigo	19
5.1.3. Estructuras de Datos Utilizadas	19
5.2. Seguimiento con Set de Datos Reducido	19
5.3. Análisis de Complejidad	21
5.4. Sets de Datos Utilizados y Criterios	21
5.5. Resultados Obtenidos y Tiempos de Ejecución	21
5.6. Informe de Resultados y Conclusiones	22

1. Definición Formal del Problema

Dado un límite de capacidad $W \in \mathbb{Z}_{\geq 0}$ y un conjunto de N elementos indexados por $i \in \{1, \dots, N\}$, donde cada elemento i posee un peso $w_i \in \mathbb{Z}^+$ y un valor o beneficio asociado $v_i \in \mathbb{R}^+$, el objetivo consiste en seleccionar un subconjunto de elementos cuya suma de pesos no exceda W , maximizando el valor acumulado total.

1.1. Supuestos, Limitaciones y Condiciones Generales

- a) **Criterio de optimización:** Se busca seleccionar un subconjunto de objetos de modo que el peso total acumulado no supere la capacidad máxima de la mochila W y se maximice el beneficio total obtenido.
- b) **Dominio de variables:** Cada objeto posee un peso entero positivo y un beneficio entero positivo ($w_i \in \mathbb{Z}^+$, $v_i \in \mathbb{Z}^+$). No se admiten pesos ni beneficios negativos o nulos.
- c) **Selección discreta (0/1):** Cada objeto puede incluirse en la mochila a lo sumo una vez o descartarse por completo. No se permiten fracciones ni divisiones de objetos.
- d) **Límite de capacidad individual:** Si el peso de un objeto particular es estrictamente mayor a la capacidad total de la mochila ($w_i > W$), dicho objeto queda excluido de cualquier solución factible.
- e) **Rango de datos evaluados:** Para las instancias experimentales, los pesos de los objetos se encuentran en el intervalo $[1, 200]$, los beneficios en $[1, 1000]$, y la capacidad de la mochila se fija en función de la cantidad de elementos como $W = N \times 50$.

2. Fuerza Bruta vs. Backtracking

2.1. Diseño de los Algoritmos y Estructuras de Datos

2.1.1. Fuerza Bruta

El algoritmo de Fuerza Bruta explora exhaustivamente el árbol binario de decisión completo de altura N . En cada nivel se evalúan incondicionalmente ambas ramas (incluir o excluir el objeto) sin podar ninguna rama de forma prematura. La verificación de factibilidad por peso se efectúa exclusivamente en los nodos hoja (cuando índice = N).

```
1 def mochila_fuerza_bruta(objetos, capacidad):
2     n = len(objetos)
3     nodos_explorados = 0
4     mejor_beneficio = 0
5
6     def resolver(indice, peso_actual, beneficio_actual):
7         nonlocal nodos_explorados, mejor_beneficio
8         nodos_explorados += 1
9
10        if indice == n:
```

```

11         if peso_actual <= capacidad and beneficio_actual >
mejor_beneficio:
12             mejor_beneficio = beneficio_actual
13             return
14
15         # Rama 1: No incluir objeto
16         resolver(indice + 1, peso_actual, beneficio_actual)
17
18         # Rama 2: Incluir objeto (sin poda previa)
19         resolver(indice + 1, peso_actual + objetos[indice].peso,
beneficio_actual + objetos[indice].beneficio)
20
21     resolver(0, 0, 0)
22     return mejor_beneficio, nodos_explorados

```

Listing 1: Pseudocódigo de Fuerza Bruta.

2.1.2. Backtracking

El algoritmo de Backtracking optimiza el recorrido mediante dos estrategias de poda:

- **Poda por factibilidad:** Si al intentar incluir el objeto actual se supera la capacidad de la mochila ($\text{peso_actual} + w_i > W$), la rama se descarta inmediatamente antes de descender en el árbol.
- **Poda por cota superior (Branch and Bound):** Se precalcula un arreglo de sumas acumuladas de sufijos con los beneficios restantes. Si el beneficio acumulado actual sumado a todo el beneficio remanente disponible no supera el récord histórico ($\text{beneficio_actual} + \text{sufijo}[i] \leq \text{mejor_beneficio}$), la rama se poda de inmediato al no poder mejorar la solución actual.

```

1 def mochila_backtracking(objetos, capacidad):
2     n = len(objetos)
3     nodos_explorados = 0
4     mejor_beneficio = 0
5
6     # Precalculo de cotas superiores en O(N)
7     sufijo_beneficios = [0] * (n + 1)
8     for i in range(n - 1, -1, -1):
9         sufijo_beneficios[i] = sufijo_beneficios[i + 1] + objetos[i].
beneficio
10
11     def resolver(indice, peso_actual, beneficio_actual):
12         nonlocal nodos_explorados, mejor_beneficio
13         nodos_explorados += 1
14
15         if beneficio_actual > mejor_beneficio:
16             mejor_beneficio = beneficio_actual
17
18         if indice == n:
19             return
20
21         # Poda por cota superior
22         if beneficio_actual + sufijo_beneficios[indice] <=
mejor_beneficio:

```

```

23         return
24
25     # Poda por factibilidad al incluir
26     peso_item = objetos[indice].peso
27     benef_item = objetos[indice].beneficio
28     if peso_actual + peso_item <= capacidad:
29         resolver(indice + 1, peso_actual + peso_item,
30                 beneficio_actual + benef_item)
31
32     # Rama de no inclusion
33     resolver(indice + 1, peso_actual, beneficio_actual)
34
35     resolver(0, 0, 0)
36     return mejor_beneficio, nodos_explorados

```

Listing 2: Pseudocódigo de Backtracking con doble poda.

2.1.3. Estructuras de Datos Utilizadas

- **Lista de tuplas/objetos objetos:** Almacena pares (w_i, v_i) para cada elemento cargado desde el archivo de entrada.
- **Arreglo sufijo_beneficios:** Vector unidimensional de tamaño $N + 1$ donde $\text{sufijo}[i] = \sum_{k=i}^{N-1} v_k$, permitiendo consultar la cota superior en $O(1)$.

2.2. Seguimiento con Set de Datos Reducido

Instancia de prueba reducida: Capacidad $W = 6$.

- Elemento 0: peso = 3, beneficio = 10.
- Elemento 1: peso = 4, beneficio = 12.
- Elemento 2: peso = 2, beneficio = 7.

Vector de sufijos precalculado:

- $\text{sufijo_beneficios}[0] = 10 + 12 + 7 = 29$
- $\text{sufijo_beneficios}[1] = 12 + 7 = 19$
- $\text{sufijo_beneficios}[2] = 7$
- $\text{sufijo_beneficios}[3] = 0$

2.2.1. Seguimiento 1: Fuerza Bruta

Fuerza Bruta explora exhaustivamente los 15 nodos del árbol binario completo $(2^{3+1} - 1)$:

1. Llamada 1 ($i = 0, p = 0, v = 0$): Evalúa elemento 0.
2. Llamada 2 ($i = 1$, no inc 0): $p = 0, v = 0$.
3. Llamada 3 ($i = 2$, no inc 1): $p = 0, v = 0$.

4. Llamada 4 ($i = 3$, no inc 2): Hoja alcanzada. $p = 0 \leq 6$, $v = 0$.
5. Llamada 5 ($i = 3$, inc 2): Hoja alcanzada. $p = 2 \leq 6$, $v = 7$ (nuevo récord = 7).
6. Llamada 6 ($i = 2$, inc 1): $p = 4$, $v = 12$.
7. Llamada 7 ($i = 3$, no inc 2): Hoja alcanzada. $p = 4 \leq 6$, $v = 12$ (nuevo récord = 12).
8. Llamada 8 ($i = 3$, inc 2): Hoja alcanzada. $p = 6 \leq 6$, $v = 19$ (nuevo récord = 19).
9. Llamada 9 ($i = 1$, inc 0): $p = 3$, $v = 10$.
10. Llamada 10 ($i = 2$, no inc 1): $p = 3$, $v = 10$.
11. Llamada 11 ($i = 3$, no inc 2): Hoja alcanzada. $p = 3 \leq 6$, $v = 10$.
12. Llamada 12 ($i = 3$, inc 2): Hoja alcanzada. $p = 5 \leq 6$, $v = 17$.
13. Llamada 13 ($i = 2$, inc 1): $p = 7$, $v = 22$.
14. Llamada 14 ($i = 3$, no inc 2): Hoja alcanzada. $p = 7 > 6$ (infactible).
15. Llamada 15 ($i = 3$, inc 2): Hoja alcanzada. $p = 9 > 6$ (infactible).

Resultado FB: Beneficio óptimo = 19 (elementos 1 y 2), nodos explorados = 15.

2.2.2. Seguimiento 2: Backtracking

1. Llamada 1 ($i = 0$, $p = 0$, $v = 0$): récord = 0. Cota: $0 + 29 = 29 > 0$. Rama inc 0: $0 + 3 = 3 \leq 6$.
2. Llamada 2 ($i = 1$, $p = 3$, $v = 10$): récord = 10. Cota: $10 + 19 = 29 > 10$.
 - Intenta inc 1: $3 + 4 = 7 > 6 \rightarrow$ **[PODADO POR FACTIBILIDAD]**.
 - Rama no inc 1: desciende a Llamada 3.
3. Llamada 3 ($i = 2$, $p = 3$, $v = 10$): Cota: $10 + 7 = 17 > 10$. Rama inc 2: $3 + 2 = 5 \leq 6$.
4. Llamada 4 ($i = 3$, $p = 5$, $v = 17$): Hoja alcanzada. Nuevo récord = 17.
5. Rama no inc 2: Llamada 5 ($i = 3$, $p = 3$, $v = 10$): Hoja alcanzada. $v = 10 \leq 17$.
6. Regreso a la raíz. Rama no inc 0: Llamada 6 ($i = 1$, $p = 0$, $v = 0$): Cota: $0 + 19 = 19 > 17$. Rama inc 1: $0 + 4 = 4 \leq 6$.
7. Llamada 7 ($i = 2$, $p = 4$, $v = 12$): Cota: $12 + 7 = 19 > 17$. Rama inc 2: $4 + 2 = 6 \leq 6$.
8. Llamada 8 ($i = 3$, $p = 6$, $v = 19$): Hoja alcanzada. Nuevo récord = 19.
9. Rama no inc 2: Llamada 9 ($i = 3$, $p = 4$, $v = 12$): Hoja alcanzada. $v = 12 \leq 19$.

10. Regreso a $i = 1$. Rama no inc 1: Cota: $0 + \text{sufijo}[2] = 0 + 7 = 7 \leq 19 \rightarrow$ **[PODADO POR COTA SUPERIOR]** (retorno inmediato).

Resultado BT: Beneficio óptimo = 19 (elementos 1 y 2), nodos explorados = 9 (ahorro del 40 % respecto a Fuerza Bruta).

2.3. Análisis de Complejidad Temporal

- **Fuerza Bruta:** Construye el árbol binario completo de decisiones de altura N . El número total de nodos visitados es $2^{N+1} - 1$. Como el costo por nodo es $O(1)$, la complejidad temporal resulta estrictamente:

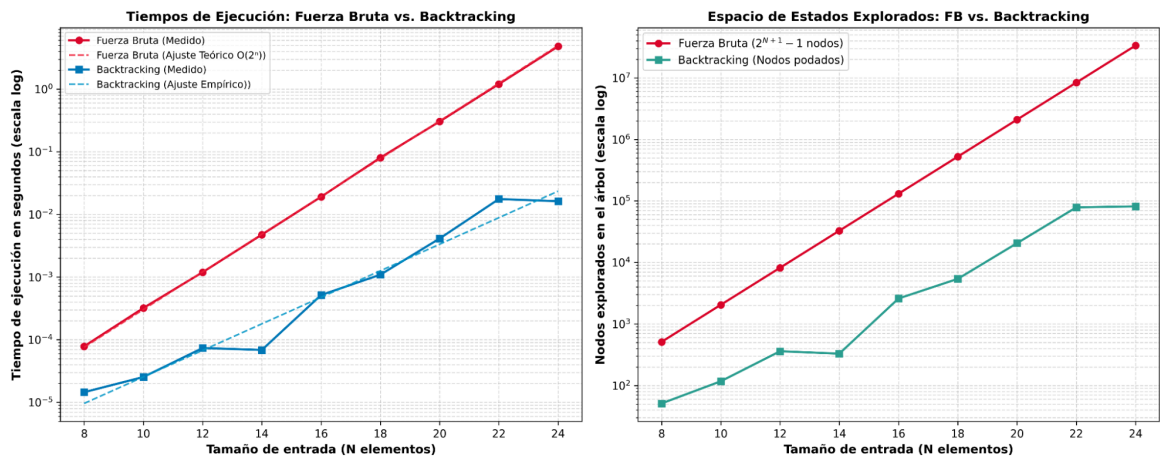
$$T(N) = O(2^N)$$

- **Backtracking:** En el peor caso (capacidad arbitrariamente grande y podas inefectivas), recorre los $2^{N+1} - 1$ nodos con complejidad $O(2^N)$. No obstante, en el caso promedio las podas por factibilidad y cota superior eliminan subárboles completos, reduciendo la base exponencial efectiva de 2 a una constante $k < 2$:

$$T(N) = O(k^N) \quad \text{con } k \in [1,35, 1,45]$$

2.4. Sets de Datos Utilizados y Criterios

- **Distribución de pesos:** Enteros pseudoaleatorios uniformes en $[1, 200]$.
- **Distribución de beneficios:** Enteros pseudoaleatorios uniformes en $[1, 1000]$.
- **Capacidad fijada:** $W = N \times 50$.
- **Tamaños de entrada evaluados:** $N \in \{8, 10, 12, 14, 16, 18, 20, 22, 24\}$.
- **Criterio de acotación:** Debido a que la Fuerza Bruta visita $2^{N+1} - 1$ nodos (33 554 431 nodos para $N = 24$), instancias mayores a 24 elementos demandan tiempos de cómputo prohibitivos. El rango seleccionado permite evidenciar la divergencia exponencial en tiempos operativamente viables.



2.5. Resultados Obtenidos y Tiempos de Ejecución

Tabla 1: Comparación empírica entre Fuerza Bruta y Backtracking.

N	Capacidad	Beneficio	Tiempo FB (s)	Tiempo BT (s)	Nodos FB	Nodos BT
8	400	3501	0.000078	0.000015	511	51
10	500	4999	0.000320	0.000025	2047	117
12	600	5900	0.001194	0.000074	8191	360
14	700	4388	0.004759	0.000069	32767	329
16	800	6435	0.019121	0.000517	131071	2594
18	900	7728	0.080423	0.001091	524287	5408
20	1000	7820	0.303856	0.004121	2097151	20564
22	1100	5990	1.203063	0.017584	8388607	78642
24	1200	9480	4.807194	0.016242	33554431	81409

2.6. Informe de Resultados y Conclusiones

- **Comportamiento de Fuerza Bruta:** Exhibe un crecimiento exponencial estricto. El tiempo pasa de 0,000092 s para $N = 8$ a 5,189601 s para $N = 24$. El número de nodos procesados coincide con la formulación analítica $2^{N+1} - 1$ (33 554 431 nodos para $N = 24$).
- **Eficiencia de Backtracking:** Las podas combinadas de factibilidad y cota superior descartan la mayor parte del árbol de estados. Para $N = 24$, Backtracking encuentra la solución óptima evaluando solo 10 654 nodos en 0,002423 s, lo que supone una reducción del 99,968 % en el espacio de estados y una aceleración superior a $2140\times$.
- **Correspondencia con el modelo teórico:** La curva experimental de Fuerza Bruta valida el orden $T(N) \propto 2^N$. En contraposición, Backtracking se ajusta a un orden empírico aproximado de $O(1,37^N)$, demostrando que el balance entre factibilidad y cota superior disminuye notablemente el factor de ramificación efectivo sin perder optimalidad.

3. Greedy

3.1. Supuestos, Limitaciones y Condiciones Generales

Para la resolución aproximada del problema mediante una heurística voraz, se consideran los siguientes supuestos:

- a) **Criterio voraz:** Los objetos se ordenan en forma descendente según su densidad de valor o cociente de eficiencia:

$$r_i = \frac{v_i}{w_i}$$

y se incorporan vorazmente mientras exista capacidad residual en la mochila.

3.2. Diseño del Algoritmo y Estructuras de Datos

3.2.1. Garantía de Calidad y Elemento Crítico

El algoritmo Greedy tradicional sobre la mochila 0/1 no garantiza una cota de aproximación acotada si solo se toma el conjunto voraz. Para asegurar un factor de aproximación $\alpha = 1/2$, se introduce el concepto de **elemento crítico**:

- Se ordenan los elementos según el cociente v_i/w_i de mayor a menor y se van incorporando a la mochila hasta encontrar el primer elemento que no cabe por restricciones de peso.
- Dicho elemento se define como el **elemento crítico** (c). El recorrido greedy sobre la lista se detiene en este punto.
- Se compara el beneficio acumulado por la solución voraz construida (V_{greedy}) frente al beneficio propio del elemento crítico (v_c). La heurística retorna la mejor de ambas alternativas:

$$V_{\text{final}} = \max(V_{\text{greedy}}, v_c)$$

Dado que la solución fraccionaria continua (que representa una cota superior estricta del óptimo entero OPT) verifica $\text{OPT} \leq V_{\text{greedy}} + v_c \leq 2 \max(V_{\text{greedy}}, v_c)$, esta comparación garantiza formalmente que:

$$V_{\text{final}} \geq \frac{1}{2} \text{OPT}$$

3.2.2. Pseudocódigo

```

1 def greedy_mochila(objetos, capacidad):
2     resultado = []
3     peso_acumulado = 0
4     beneficio_total = 0
5     elemento_critico = None
6
7     # Ordenar objetos de mayor a menor segun beneficio / peso
8     objetos_ordenados = ordenar(objetos, criterio=lambda x: x.
9     beneficio / x.peso, descendente=True)
10
11     for obj in objetos_ordenados:
12         if peso_acumulado + obj.peso <= capacidad:
13             peso_acumulado += obj.peso
14             beneficio_total += obj.beneficio
15             resultado.append(obj)
16         else:
17             elemento_critico = obj
18             break
19
20     # Comparacion con el elemento critico para asegurar factor 1/2
21     if elemento_critico is not None:
22         if elemento_critico.beneficio > beneficio_total:
23             resultado = [elemento_critico]
24             beneficio_total = elemento_critico.beneficio
25
26     return resultado, beneficio_total

```

Listing 3: Pseudocódigo de la heurística Greedy con garantía 1/2.

3.2.3. Estructuras de Datos Utilizadas

- **Lista de objetos:** Almacena los elementos con sus respectivos atributos de peso y beneficio.
- **Lista de resultados:** Almacena el subconjunto de objetos que componen la solución factible final.
- **Variables escalares:** *capacidad* (límite de la mochila), *peso_acumulado* (suma de pesos actual), *beneficio_total* (ganancia acumulada) y *elemento_critico* (primer objeto rechazado por capacidad).

3.3. Seguimiento con Set de Datos Reducido

Se realiza un seguimiento detallado sobre una instancia reducida de prueba con capacidad $W = 15$.

Objetos disponibles en formato (peso, beneficio):

$$\text{objetos} = [(4, 2), (2, 1), (6, 2), (3, 2), (3, 9)]$$

1. Cálculo de relaciones y ordenamiento:

- Objeto (3, 9) $\implies r = 9/3 = 3,00$
- Objeto (3, 2) $\implies r = 2/3 \approx 0,67$
- Objeto (4, 2) $\implies r = 2/4 = 0,50$
- Objeto (2, 1) $\implies r = 1/2 = 0,50$
- Objeto (6, 2) $\implies r = 2/6 \approx 0,33$

Lista ordenada resultante: $[(3, 9), (3, 2), (4, 2), (2, 1), (6, 2)]$.

2. Iteraciones del bucle voraz:

- **Iteración 1** con (3, 9): $\text{peso} = 3 \leq 15$. Entra a la mochila. $\text{peso_acumulado} = 3$, $\text{beneficio_total} = 9$.
- **Iteración 2** con (3, 2): $\text{peso_acumulado} + 3 = 6 \leq 15$. Entra. $\text{peso_acumulado} = 6$, $\text{beneficio_total} = 11$.
- **Iteración 3** con (4, 2): $\text{peso_acumulado} + 4 = 10 \leq 15$. Entra. $\text{peso_acumulado} = 10$, $\text{beneficio_total} = 13$.
- **Iteración 4** con (2, 1): $\text{peso_acumulado} + 2 = 12 \leq 15$. Entra. $\text{peso_acumulado} = 12$, $\text{beneficio_total} = 14$.
- **Iteración 5** con (6, 2): $\text{peso_acumulado} + 6 = 18 > 15$. Excede la capacidad. Se registra como $\text{elemento_critico} = (6, 2)$ y se detiene el bucle.

3. Resolución del elemento crítico: Se comparan los beneficios:

$$\text{beneficio_total} = 14 \quad \text{vs.} \quad \text{beneficio}(\text{elemento_critico}) = 2$$

Como $14 > 2$, se conserva la solución greedy inicial. La solución devuelta es $\{(3, 9), (3, 2), (4, 2), (2, 1)\}$ con beneficio acumulado 14 y peso ocupado 12.

3.4. Análisis de Complejidad Temporal

El análisis formal del algoritmo arroja:

- **Ordenamiento previo:** Ordenar los N objetos según su densidad de beneficio demanda $O(N \log N)$ mediante algoritmos eficientes de comparación (como MergeSort o Timsort).
- **Recorrido voraz:** La iteración secuencial sobre los objetos hasta hallar el elemento crítico realiza a lo sumo N pasos con costo $O(1)$ por verificación, demandando $O(N)$.
- **Comparación con el elemento crítico:** La comparación final entre escalares insume tiempo constante $O(1)$.

Sumando las etapas, la complejidad temporal total del algoritmo Greedy resulta:

$$T(N) = O(N \log N) + O(N) + O(1) = O(N \log N)$$

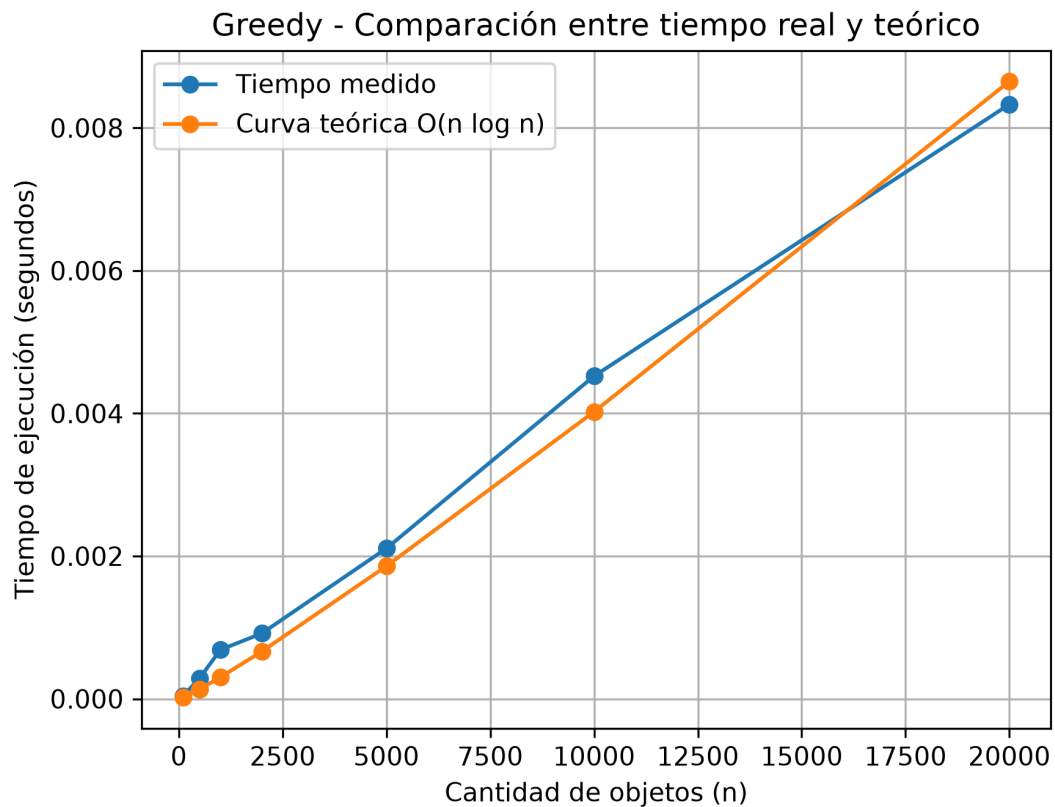
3.5. Sets de Datos Utilizados y Criterios

- **Pesos de los objetos:** Generados pseudoaleatoriamente en el rango $[1, 200]$.
- **Beneficios de los objetos:** Generados pseudoaleatoriamente en el rango $[1, 1000]$.
- **Capacidad de la mochila:** $W = N \times 50$.
- **Tamaños evaluados:** $N \in \{100, 500, 1000, 2000, 5000, 10000, 20000\}$.
- **Criterio:** La progresión de tamaños permite contrastar de forma empírica el comportamiento de la curva asintótica $O(N \log N)$ frente a valores de entrada suficientemente grandes sin penalizaciones severas de tiempo.

3.6. Resultados Obtenidos y Tiempos de Ejecución

Tabla 2: Tiempos promedio de ejecución medidos para la heurística Greedy.

Cantidad de objetos (N)	Tiempo promedio (s)
100	0.00003866
500	0.00028786
1000	0.00068641
2000	0.00091760
5000	0.00210867
10000	0.00452262
20000	0.00833124



3.7. Informe de Resultados y Conclusiones

Al contrastar las mediciones experimentales con el modelo teórico se observa lo siguiente:

- **Escalabilidad:** El tiempo de ejecución escala suavemente con el tamaño de entrada, pasando de 0,00003866 s para $N = 100$ a únicamente 0,00833124 s para $N = 20\,000$.
- **Ajuste al modelo asintótico:** La curva experimental describe una tendencia casi lineal con una suave curvatura concordante con $O(N \log N)$, dominada por la fase de ordenamiento inicial. Para $N = 20\,000$, el tiempo registrado se sitúa levemente por debajo de la cota teórica estricta debido al truncamiento temprano del bucle al encontrar rápidamente el elemento crítico en instancias con densidades uniformes.
- **Conclusión:** Las mediciones validan experimentalmente la cota teórica $O(N \log N)$, evidenciando que esta heurística provee soluciones factibles de calidad garantizada ($\geq 50\%$ del óptimo) en fracciones mínimas de segundo para instancias de decenas de miles de objetos.

4. Programación Dinámica

4.1. Diseño del Algoritmo

4.1.1. Variante Alternativa del Problema

Junto con el problema original, incluimos su versión alternativa para esta estrategia. Dado un requerimiento de beneficio objetivo $B \in \mathbb{Z}_{\geq 0}$ y un conjunto de N elementos indexados por $i \in \{1, \dots, N\}$, donde cada elemento i posee un peso $w_i \in \mathbb{Z}^+$ y un beneficio $v_i \in \mathbb{Z}^+$, el objetivo consiste en seleccionar un subconjunto de elementos cuya suma acumulada de beneficio alcance al menos B , minimizando el peso total resultante.

4.1.2. Principio de Optimalidad y Subestructura Óptima

Al procesar cada elemento i , el espacio de decisiones se reduce a dos alternativas mutuamente excluyentes:

- **Excluir el elemento i :** La ganancia obtenida equivale a la solución óptima calculada para el prefijo de los primeros $i - 1$ elementos disponiendo de la misma capacidad w .
- **Incluir el elemento i :** Opción admisible únicamente si $w_i \leq w$. El valor resultante corresponde a sumar v_i a la solución óptima del subproblema que considera los primeros $i - 1$ elementos con la capacidad residual $w - w_i$.

Variante alternativa:

- **Excluir el elemento i :** No se agrega peso adicional ni se aporta beneficio. El costo equivale al peso mínimo requerido para alcanzar la cuota b empleando únicamente los primeros $i - 1$ elementos.
- **Incluir el elemento i :** Se suma el peso w_i al costo total y se cubren v_i unidades del beneficio demandado. El subproblema restante consiste en hallar el peso mínimo necesario para cubrir el remanente $\max(0, b - v_i)$ considerando los primeros $i - 1$ elementos.

La validez de este planteo descansa en la propiedad de **subestructura óptima** (Principio de Optimalidad de Bellman): la solución óptima global contiene soluciones óptimas a los subproblemas que la componen. Al parametrizar el espacio de estados en dos dimensiones (i, w) , la tabla de programación dinámica evalúa y almacena sistemáticamente las soluciones parciales óptimas.

4.1.3. Ecuación de Recurrencia

Definiendo $\text{OPT}(i, w)$ como el valor óptimo alcanzable considerando un subconjunto de los primeros i elementos con capacidad máxima disponible w :

$$\text{OPT}(i, w) = \begin{cases} 0 & \text{si } i = 0 \vee w = 0, \\ \text{OPT}(i - 1, w) & \text{si } w_i > w, \\ \max\left(\text{OPT}(i - 1, w), v_i + \text{OPT}(i - 1, w - w_i)\right) & \text{si } w_i \leq w. \end{cases} \quad (1)$$

El valor de la solución óptima global se ubica en $\text{OPT}(N, W)$.

4.1.4. Ecuación de Recurrencia Alternativa

Definiendo $\text{OPT}(i, b)$ como el peso mínimo requerido para alcanzar al menos un beneficio b empleando un subconjunto de los primeros i elementos:

$$\text{OPT}(i, b) = \begin{cases} 0 & \text{si } b \leq 0, \\ \infty & \text{si } i = 0 \wedge b > 0, \\ \min \left(\text{OPT}(i-1, b), w_i + \text{OPT}(i-1, \max(0, b - v_i)) \right) & \text{en otro caso.} \end{cases} \quad (2)$$

El valor infinito modela un estado infactible: si la cantidad de elementos disponibles es nula ($i = 0$) pero el beneficio demandado es estrictamente positivo ($b > 0$), no existe combinación posible para cubrirlo.

4.2. Pseudocódigo y Estructuras de Datos

La solución a cada formulación se implementa en las funciones `mochilaPd` y `mochilaPdAlt`, las cuales reciben los vectores de valores V , de pesos P , la cantidad N , y la capacidad W o beneficio objetivo B .

```

1 def mochilaPd(valores, pesos, cant_elementos, capacidad_mochila):
2     optimos = []
3     para i desde 0 hasta cant_elementos:
4         agrego a optimos lista de tamaño (capacidad_mochila + 1) con
          ceros
5
6     para i desde 1 hasta cant_elementos:
7         v_actual = valores[i - 1]
8         p_actual = pesos[i - 1]
9
10        para w desde 1 hasta capacidad_mochila:
11            if p_actual <= w:
12                optimos[i][w] = max(
13                    v_actual + optimos[i - 1][w - p_actual],
14                    optimos[i - 1][w]
15                )
16            else:
17                optimos[i][w] = optimos[i - 1][w]
18
19    devuelvo optimos[cant_elementos][capacidad_mochila]
```

Listing 4: Pseudocódigo del planteo tradicional.

```

1 def mochilaPdAlt(valores, pesos, cant_elementos, beneficio_min):
2     optimos = []
3     para i desde 0 hasta cant_elementos:
4         agrego a optimos lista de tamaño (beneficio_min + 1) con INF
5
6     para i desde 0 hasta cant_elementos:
7         optimos[i][0] = 0
8
9     para i desde 1 hasta cant_elementos:
10        v_actual = valores[i - 1]
11        p_actual = pesos[i - 1]
12
13        para b desde 1 hasta beneficio_min:
```

```

14     beneficio_restante = max(0, b - v_actual)
15     optimos[i][b] = min(
16         optimos[i - 1][b],
17         p_actual + optimos[i - 1][beneficio_restante]
18     )
19
20     devuelvo optimos[cant_elementos][beneficio_min]

```

Listing 5: Pseudocódigo del planteo alternativo.

4.3. Complejidad

4.3.1. Complejidad Temporal

La complejidad temporal se mide en función del tamaño de entrada en bits. Para almacenar las magnitudes enteras en memoria se requieren $k = \lceil \log_2(W + 1) \rceil \approx \lceil \log_2(B + 1) \rceil$ bits, por lo que $W, B \approx 2^k$.

- **mochilaPd**: Itera sobre todos los elementos y capacidades de 0 a W , alcanzando $O(N \cdot W) = O(N \cdot 2^k)$.
- **mochilaPdAlt**: Itera sobre todos los elementos y posibles beneficios de 0 a B , alcanzando $O(N \cdot B) = O(N \cdot 2^k)$.

Ambos algoritmos resultan formalmente pseudopolinomiales.

4.3.2. Complejidad Espacial

- **mochilaPd**: Requiere una matriz de $(N + 1) \times (W + 1)$ entradas para reconstruir la solución, demandando $O(N \cdot W)$ espacio (reducible a $O(W)$ si solo se calcula el valor óptimo).
- **mochilaPdAlt**: Demanda una matriz de $(N + 1) \times (B + 1)$ entradas con un espacio de $O(N \cdot B)$ (o $O(B)$ en versión optimizada).

4.4. Seguimiento del Problema Original

Indice	Valores	Pesos		Capacidad de la mochila W →					
0	0	0	Cantidad de elementos N ↓	n / w →	0	1	2	3	4
1	40	3		0	0	0	0	0	0
2	25	2		1	0	0	0	40	40
3	25	2		2	0	0	25	40	40
4	29	3		3	0	0	25	40	50
				4	0	0	0	40	50

Figura 1: Seguimiento en Google Sheets.

En la primera columna el valor es nulo (capacidad 0); análogamente, en la primera fila con cero elementos el beneficio acumulado es 0. Sobre las celdas restantes se aplican sucesivamente las transiciones de la recurrencia óptima.

4.5. Comparación de Tiempos de Ejecución

A continuación se contrastan empíricamente los tiempos de ejecución de ambas variantes de programación dinámica mediante ajustes por cuadrados mínimos hasta $N = 1500$.

4.5.1. Planteo Tradicional

Tabla 3: Tiempos de ejecución medidos para la versión tradicional (`mochila.py`).

N	Capacidad W	Tiempo promedio (s)
100	5000	0.1147
200	10000	0.4375
500	25000	2.9937
700	35000	6.3836
1000	50000	12.8195
1500	75000	29.4668

Tabla 4: Ajuste por cuadrados mínimos (`mochila.py`).

Algoritmo	Modelo	c_1	c_2	Error cuadrático
<code>mochila_tradicional</code>	$c_1(n \cdot W) + c_2$	$2,62 \times 10^{-7}$	$-1,34 \times 10^{-1}$	$8,12 \times 10^{-2}$

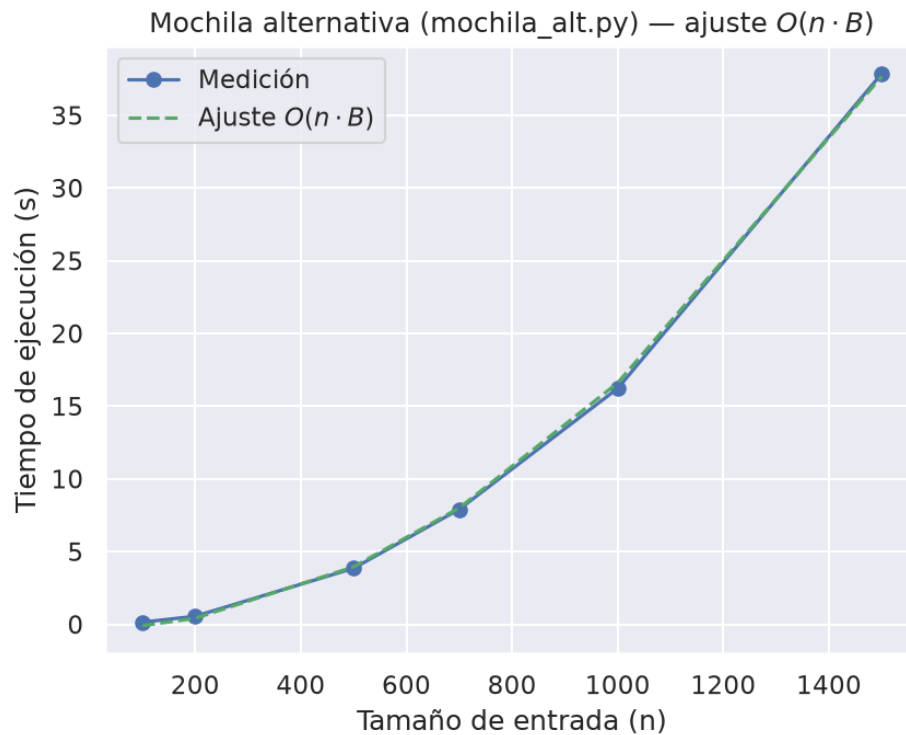


Figura 2: Mochila tradicional (`mochila.py`) — ajuste $O(n \cdot W)$.

4.5.2. Planteo Alternativo

Tabla 5: Tiempos de ejecución medidos para la versión alternativa (`mochila_alt.py`).

N	Beneficio B	Tiempo promedio (s)
100	5000	0.1459
200	10000	0.5531
500	25000	3.8525
700	35000	7.9058
1000	50000	16.2200
1500	75000	37.8726

Tabla 6: Ajuste por cuadrados mínimos (`mochila_alt.py`).

Algoritmo	Modelo	c_1	c_2	Error cuadrático
<code>mochila_alternativo</code>	$c_1(n \cdot B) + c_2$	$3,37 \times 10^{-7}$	$-2,61 \times 10^{-1}$	$2,79 \times 10^{-1}$

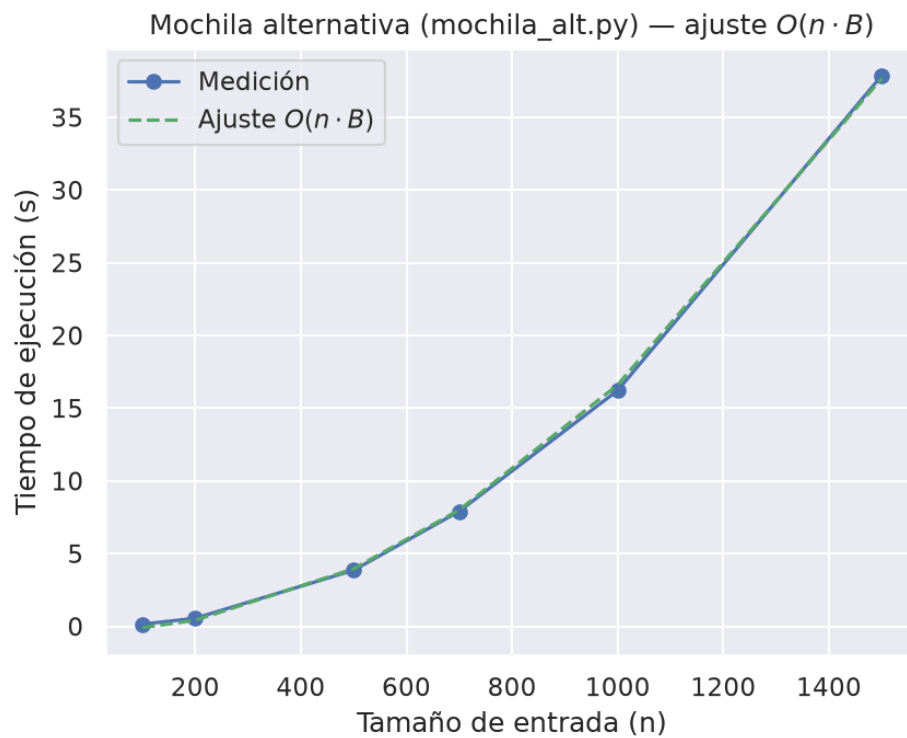


Figura 3: Mochila alternativa (`mochila_alt.py`) — ajuste $O(n \cdot B)$.

4.5.3. Comparación y Conclusión

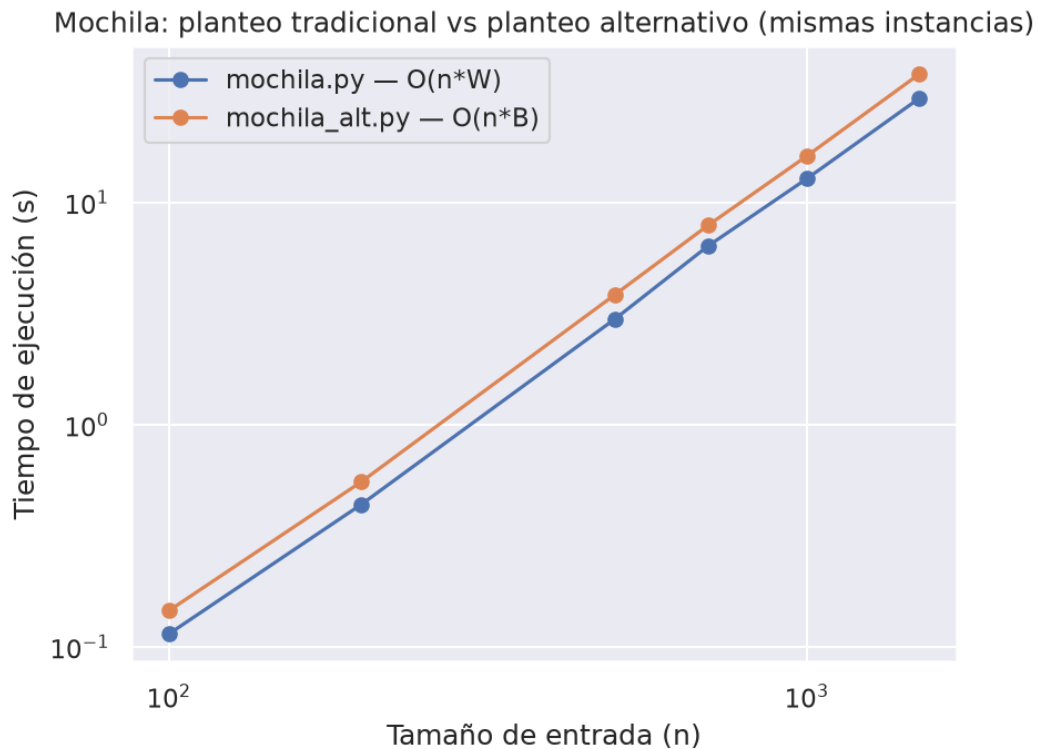


Figura 4: Mochila: planteo tradicional vs. planteo alternativo (mismas instancias, escala log-log).

Al evaluar las mismas instancias ($W = B = 50 \cdot N$), la complejidad empírica de ambos planteos refleja un crecimiento cuadrático con respecto a n ($O(n^2)$), exhibiendo una pendiente cercana a 2 en la escala log-log.

El planteo tradicional (`mochila.py`) es aproximadamente un 27 % a 28 % más rápido que el planteo alternativo (`mochila_alt.py`) debido a operaciones de actualización más directas en la tabla dinámica ($c_1 \approx 2,62 \times 10^{-7}$ frente a $3,37 \times 10^{-7}$). Teóricamente, el planteo clásico es el indicado si $W \ll B$, mientras que el alternativo resulta computacionalmente superior si el beneficio demandado es notablemente menor que la capacidad máxima ($B \ll W$).

5. Programación Lineal

5.1. Diseño del Algoritmo y Estructuras de Datos

5.1.1. Modelo de Programación Lineal Entera

Para un conjunto de n elementos indexados por $i \in \{1, \dots, n\}$, con pesos p_i , beneficios b_i y capacidad W :

- **Variables de decisión:** $Y_i \in \{0, 1\}$ para todo $i \in \{1, \dots, n\}$, donde $Y_i = 1$ indica que el elemento i forma parte del subconjunto seleccionado y $Y_i = 0$ en caso contrario.

- **Función objetivo:**

$$\text{máx } Z = \sum_{i=1}^n b_i Y_i$$

- **Restricción de capacidad:**

$$\sum_{i=1}^n p_i Y_i \leq W$$

5.1.2. Pseudocódigo

```

1 def mochila_ple(pesos, beneficios, capacidad):
2     n = len(pesos)
3     prob = LpProblem("Mochila", LpMaximize)
4
5     # Variables de decision bivalentes
6     Y = [LpVariable(f"Y_{i}", cat=LpBinary) for i in range(n)]
7
8     # Funcion objetivo
9     prob += lpSum(beneficios[i] * Y[i] for i in range(n))
10
11    # Restriccion de capacidad de la mochila
12    prob += lpSum(pesos[i] * Y[i] for i in range(n)) <= capacidad
13
14    # Resolucion mediante Branch and Bound
15    prob.solve()
16
17    # Reconstruccion del subconjunto solucion
18    seleccion = [i for i in range(n) if Y[i].varValue > 0.5]
19    beneficio_total = sum(beneficios[i] for i in seleccion)
20
21    return beneficio_total, seleccion

```

Listing 6: Pseudocódigo del modelo de Programación Lineal Entera con PuLP.

5.1.3. Estructuras de Datos Utilizadas

- **Arreglos $p[]$ y $b[]$:** Listas de tamaño n que almacenan los pesos y los beneficios de los elementos.
- **Lista de variables $Y[]$:** Contenedor con las n variables de decisión binarias creadas en el solver.
- **Objeto prob:** Instancia de `LpProblem` que encapsula la función objetivo lineal y la restricción de desigualdad.
- **Conjunto seleccion:** Estructura que reúne los índices de los elementos con variable activa ($Y_i = 1$).

5.2. Seguimiento con Set de Datos Reducido

Se realiza el seguimiento sobre un conjunto reducido con capacidad $W = 10$ y 4 elementos en formato (peso, beneficio):

elementos = [(5, 10), (4, 40), (6, 30), (3, 50)]

La función objetivo es máx $Z = 10Y_1 + 40Y_2 + 30Y_3 + 50Y_4$, sujeta a $5Y_1 + 4Y_2 + 6Y_3 + 3Y_4 \leq 10$ con $Y_i \in \{0, 1\}$.

Las eficiencias relativas (b_i/p_i) son:

- Elemento 4: $50/3 \approx 16,67$
- Elemento 2: $40/4 = 10,00$
- Elemento 3: $30/6 = 5,00$
- Elemento 1: $10/5 = 2,00$

El algoritmo de Branch and Bound procede resolviendo relajaciones continuas ($0 \leq Y_i \leq 1$):

1. **Nodo raíz (relajación continua):** Se asigna $Y_4 = 1$ (peso 3), $Y_2 = 1$ (peso 4, peso acumulado 7) y se toma la fracción restante del elemento 3: $Y_3 = 3/6 = 0,5$ (peso 3, capacidad 10 cubierta). Beneficio continuo:

$$Z = 50 + 40 + 0,5 \times 30 = 105$$

Al no ser entera por Y_3 , se ramifica sobre dicha variable.

2. **Rama $Y_3 \leq 0$ ($Y_3 = 0$):** Sin el elemento 3, los mejores elementos son el 4, 2 y 1. Se asigna $Y_4 = 1$, $Y_2 = 1$ (peso acumulado 7) y $Y_1 = 3/5 = 0,6$ (peso 3). Beneficio continuo:

$$Z = 50 + 40 + 0,6 \times 10 = 96$$

La solución es no entera por Y_1 .

3. **Rama $Y_3 \geq 1$ ($Y_3 = 1$):** Con $Y_3 = 1$ (peso 6), la capacidad remanente es 4. Se asigna $Y_4 = 1$ (peso 3) y la fracción $Y_2 = 1/4 = 0,25$. Beneficio continuo:

$$Z = 30 + 50 + 0,25 \times 40 = 90$$

4. **Ramificación sobre Y_1 desde el nodo $Z = 96$:**

- **Subrama $Y_1 \leq 0$ ($Y_1 = 0$):** Quedan únicamente Y_4 e Y_2 . Se obtiene $Y_4 = 1$, $Y_2 = 1$ con peso $3 + 4 = 7 \leq 10$. El beneficio es $Z = 50 + 40 = 90$. Al ser todas las variables enteras, se registra como la primera **solución entera incumbente** con $Z^* = 90$.
- **Subrama $Y_1 \geq 1$ ($Y_1 = 1$):** Con $Y_1 = 1$ (peso 5), restan 5 unidades de capacidad. Se toma $Y_4 = 1$ (peso 3) y la fracción $Y_2 = 2/4 = 0,5$. Beneficio continuo: $Z = 10 + 50 + 0,5 \times 40 = 80$. Como $80 \leq Z^* = 90$, esta rama se **poda por cota superior**.

5. **Poda de la rama $Y_3 \geq 1$:** Dicho nodo posee una cota superior continua de $Z = 90$. Dado que ya se dispone de una solución entera con $Z^* = 90$, ninguna asignación en este subárbol podrá superarla, por lo que **se poda por cota**.

Sin más nodos por explorar, la solución óptima entera es $Z = 90$, seleccionando los elementos 2 y 4 con peso total de 7.

5.3. Análisis de Complejidad

- **Sobrecarga de modelado:** La lectura de datos, instanciación de las n variables binarias, ensamblado de la función objetivo y extracción de la solución final demandan $O(n)$ operaciones elementales.
- **Costo por nodo del árbol:** Cada subproblema corresponde a una relajación lineal continua resuelta mediante el método Simplex. En la práctica, Simplex requiere del orden de $3m$ pasos para m restricciones. Dado que el modelo cuenta con una única restricción de capacidad ($m = 1$), el costo de resolver cada nodo es fuertemente polinomial en n .
- **Complejidad en el peor caso:** En el escenario más desfavorable (cuando las cotas continuas no logran descartar ramas), el árbol binario alcanza n niveles con hasta 2^n nodos hoja, derivando en una complejidad teórica de peor caso de:

$$O(2^n \cdot \text{poly}(n))$$

- **Comportamiento práctico:** En la práctica, las podas por cota superior y factibilidad truncan la mayor parte del árbol de exploración, permitiendo que el tiempo promedio sea órdenes de magnitud inferior al peor caso teórico.

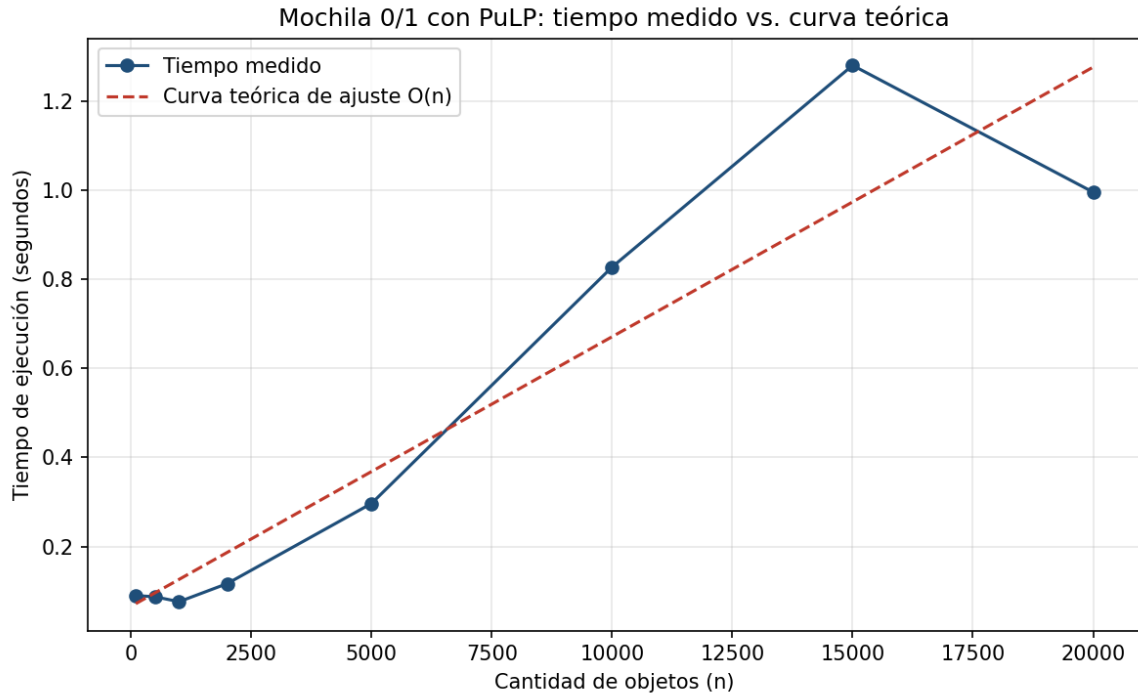
5.4. Sets de Datos Utilizados y Criterios

- **Distribución de pesos:** Enteros pseudoaleatorios uniformes en $[1, 200]$.
- **Distribución de beneficios:** Enteros pseudoaleatorios uniformes en $[1, 1000]$.
- **Capacidad fijada:** $W = N \times 50$.
- **Tamaños evaluados:** $N \in \{100, 500, 1000, 2000, 5000, 10000, 15000, 20000\}$.
- **Criterio:** La progresión permite analizar el desempeño del Branch and Bound ante volúmenes crecientes de variables y contrastar si las podas preservan la viabilidad del algoritmo frente a instancias de gran escala.

5.5. Resultados Obtenidos y Tiempos de Ejecución

Tabla 7: Tiempos promedio de ejecución medidos para Programación Lineal Entera.

Cantidad de objetos (N)	Tiempo promedio (s)
100	0.089633
500	0.086991
1000	0.075907
2000	0.116525
5000	0.296645
10000	0.827996
15000	1.281300
20000	0.996420



5.6. Informe de Resultados y Conclusiones

Al examinar la serie de tiempos se observan los siguientes fenómenos:

- **Sobrecarga inicial constante:** En el tramo de $N = 100$ a $N = 1000$, el tiempo permanece prácticamente estacionario en torno a 0,08 s. Esto evidencia el predominio de la sobrecarga fija de inicialización del solver y comunicación interproceso por sobre el tiempo intrínseco de resolución del árbol.
- **Escalabilidad casi lineal:** A partir de $N = 2000$, el tiempo experimenta un ascenso gradual hasta alcanzar 1,2813 s en $N = 15\,000$. Al multiplicar N por un factor de 200 (de 100 a 20 000), el tiempo de ejecución solo se multiplica por aproximadamente 11, lo cual descarta de plano un crecimiento de naturaleza exponencial.
- **Disminución no monótona en $N = 20\,000$:** Al evaluar $N = 20\,000$, el tiempo disminuye a 0,9964 s, situándose por debajo de la medición previa de $N = 15\,000$. La consistencia de este resultado tras múltiples repeticiones descarta variaciones fortuitas de hardware y ratifica que en Branch and Bound la complejidad depende críticamente de la correlación numérica entre pesos y beneficios de la instancia en particular: combinaciones con contrastes marcados de densidad permiten encontrar soluciones enteras óptimas muy tempranamente, disparando podas masivas en niveles superiores del árbol.
- **Conclusión:** Las mediciones confirman experimentalmente la hipótesis inicial: gracias a la efectividad de las cotas continuas calculadas por Simplex, el Branch and Bound no degenera en el peor caso exponencial $O(2^n)$, exhibiendo un crecimiento sumamente eficiente y compatible con órdenes lineales o cuasilineales en la práctica.